

Lab 11: Ordinary Differential Equations

Vladyslav Humeniuk

1 Introduction

The objective of this laboratory was to explore numerical methods for solving Ordinary Differential Equations (ODEs) formulated as Initial Value Problems (IVPs). Numerical solutions approximate the continuous differential equation by replacing it with discrete algebraic steps [2]. This report investigates two primary aspects of ODE solvers using the GNU Scientific Library (GSL): the impact of step-size control strategies (adaptive versus fixed steps) on complex oscillatory systems, and the comparative accuracy of the fundamental Euler method versus the higher-order Runge-Kutta algorithm [1].

2 Methodology

The computational tasks were divided into two distinct parts:

Part A: The van der Pol Oscillator The non-linear van der Pol equation, $u''(t) + \mu u'(t)(u(t)^2 - 1) + u(t) = 0$, was solved by reducing it to a system of two first-order ODEs:

$$\begin{aligned}u' &= v \\v' &= -u + \mu v(1 - u^2)\end{aligned}$$

Three different GSL implementations were analyzed to understand step-size management:

1. `ode-ex1.c`: Utilizes a high-level driver with an adaptive Runge-Kutta Prince-Dormand (8,9) method (`rk8pd`). It forces evenly spaced outputs, hiding the internal adaptive micro-steps.
2. `ode-ex2.c`: Utilizes a low-level stepper interface (`gsl_odeiv2_evolve`) to output the system state at every single adaptive internal step taken by the algorithm.
3. `ode-ex3.c`: Utilizes a high-level driver but enforces strictly fixed internal steps using the classic 4th-order Runge-Kutta method (`rk4`).

Part B: Method Accuracy Comparison An initial value problem, $y' + y \cos(x) = \sin(x) \cos(x)$ with $y(0) = 0$, was transformed to the standard explicit form $y' = \sin(x) \cos(x) - y \cos(x)$. The equation was solved numerically on the interval $[0, 15]$ using two approaches: the standard explicit Euler method and the 4th-order Runge-Kutta method (via GSL). To accurately evaluate the numerical drift, the results were compared against the exact analytical solution: $y_{exact}(x) = \sin(x) - 1 + e^{-\sin(x)}$.

3 Results

3.1 Part A: Step Distribution Analysis

The solutions to the van der Pol equation ($\mu = 10$) are visualized in Figure 1. The 3x1 multiplot explicitly demonstrates the differences in how the algorithms navigate the time domain.

- **Top Plot (Ex 1):** The adaptive solver successfully integrates the stiff equation and outputs exactly 100 evenly spaced points. The complexity of the internal calculations is abstracted away from the user.
- **Middle Plot (Ex 2):** By exposing the low-level stepper, the true nature of adaptive solvers is revealed. The algorithm takes massive leaps where the function is smooth and relatively flat, but clusters a dense grouping of tiny micro-steps in regions of high gradient (rapid changes in $u(t)$) to maintain precision and stability without wasting computational resources.
- **Bottom Plot (Ex 3):** The fixed-step solver produces the same curve but does so by brute-forcing the problem. It executes exactly 1000 identical micro-steps between each output point, regardless of the local gradient, which is computationally inefficient for varying functions.

3.2 Part B: Runge-Kutta vs. Euler Method

Figure 2 illustrates the comparative accuracy of the Runge-Kutta 4 and Euler methods against the exact analytical solution.

The explicit Euler method, being a first-order algorithm, evaluates the derivative only at the beginning of the step. As observed in the plot, this leads to significant truncation error. The Euler trajectory consistently overshoots or undershoots the true curve, particularly at the peaks and troughs where the derivative changes direction rapidly. The error accumulates over time, resulting in a noticeable phase and amplitude drift.

Conversely, the 4th-order Runge-Kutta method evaluates the derivative at four different points within each step (including midpoints) to calculate a weighted average. This effectively cancels out lower-order error terms, resulting in a curve that perfectly overlays the exact analytical solution throughout the entire domain.

4 Conclusion

The laboratory experiments successfully demonstrated the critical principles of numerical ODE solving. Part A highlighted that adaptive step-size control is vastly superior to fixed-step integration for non-linear equations, as it automatically balances high precision in turbulent regions with computational efficiency in smooth regions. Part B visually confirmed that higher-order methods, such as Runge-Kutta, are strictly necessary for scientific computing, as the basic Euler method is highly susceptible to accumulated truncation errors, especially in highly oscillatory systems.

Bibliography

- [1] GNU Project: GNU Scientific Library Reference Manual: Ordinary Differential Equations (2024), <https://www.gnu.org/software/gsl/doc/html/ode-initval.html>, accessed strictly for GSL ODE routines
- [2] Heath, M.T.: Scientific computing: An introductory survey, chapter 9 - initial value problems for ordinary differential equations. Department of Computer Science, University of Illinois at Urbana-Champaign (2002), http://artemis.wszib.edu.pl/~funika/mois/lab11/heath_ODE.pdf

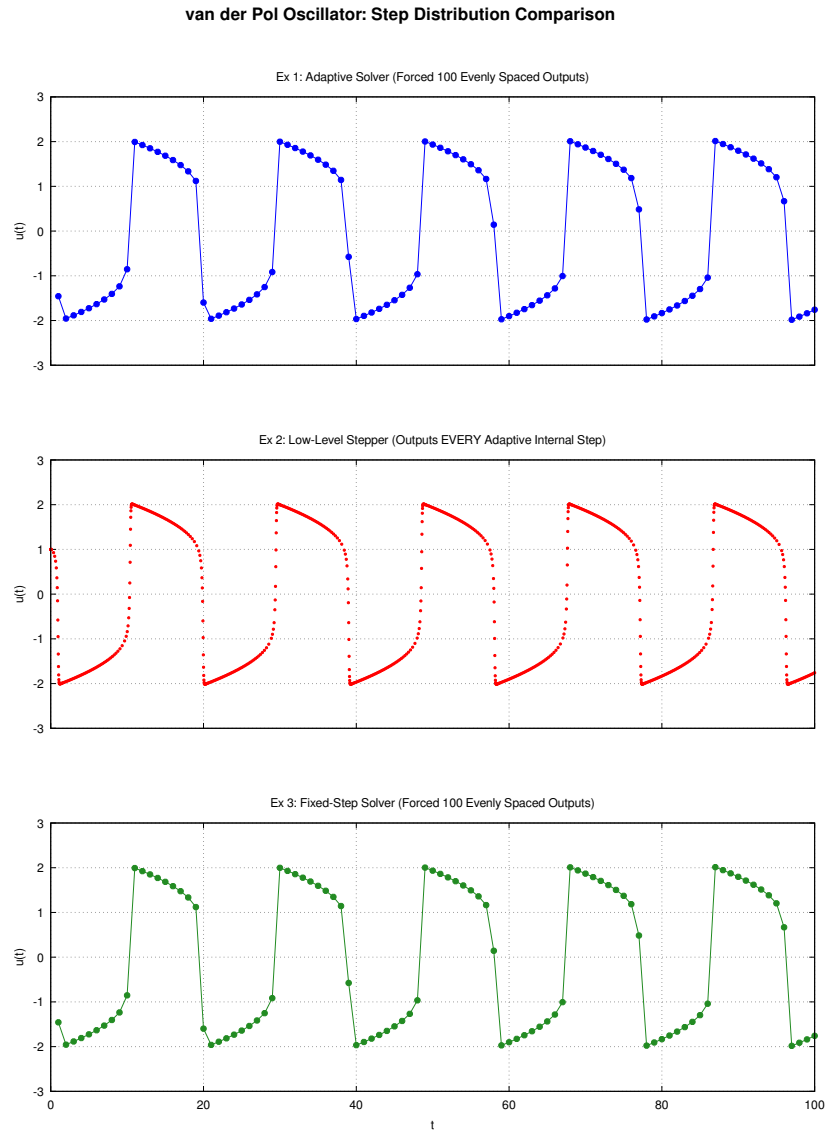


Figure 1: Step distribution comparison for the van der Pol oscillator.

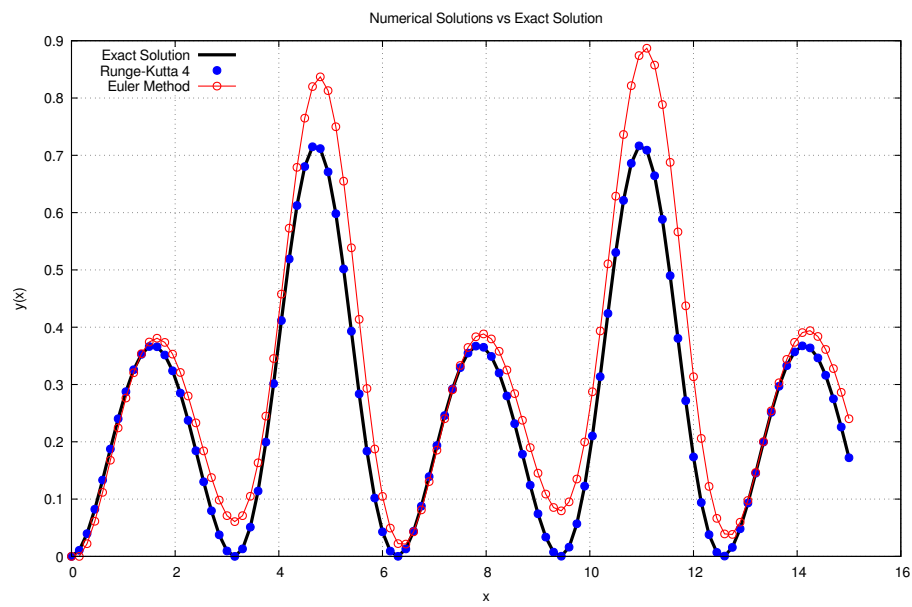


Figure 2: Numerical solutions vs. exact solution for $y' = \sin(x) \cos(x) - y \cos(x)$.